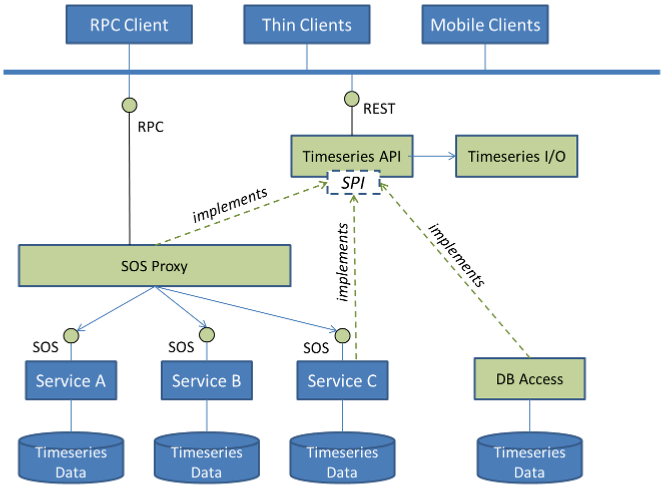
General
Collections
Stations
Timeseries
Styles
Data Formats
Generalization
Parameters
Unstable/Legacy

About

The 52°North Timeseries API provides a thin access layer to timeseries data via RESTful Web binding. It provides different output formats like json, pdf or png.

This API development intends to make near realtime sensor data (provided by OGC Sensor Observation Services) better accessible for leightweight clients like mobile devices. However, in addition to the actual Web access layer the Timeseries API offers a Service Provider Interface (SPI) which can be implemented to adapt arbitrary data sources.

Overview



Document status

What	When
Added raw data output for timeseries and procedures.	2016-01-06
Added extras metadata.	2015-12-14
Added generalization section.	2015-07-06
Added station to general query parameters.	2014-02-06
Create legacy section and add low-level SOS concepts.	2013-12-16

General

top

Content-Types

The API provides different content types which can be used to retrieve different types of representation of a requested resource. The actual content types differ for each resource so the ones supported are documented in the appropriate section.

The only content type supported by each resource is `application/json` . It is set as the default content type but can be set either via HTTP header or by `.json` extension.

Common Query Parameters

Each parameter is optional and can only contain one value. Multiple parameters are combined to an `AND` query. A resource may specify further query parameters to control/filter output. Special query parameters are described in each section.

Parameter	Example	Description
`expanded`	`expanded=true`	If `true` a list of partially expanded resource items is returned. The expansion provides more detailed information up to a certain point. However, it does not necessarily provide all item information as one may get when request the resource item itself. The `expanded` works almost only for resource collection. However, exceptions may exist (e.g. for timeseries raw data). Exceptions are explicitly documented. The default is `false` .
`service`	`service=srv_sdlfj2`	All resources where the given service id does match.
`category`	`category=cat_sadf2d`	All resources where the given category id does match.
`statiopn`	`station=sta_sadf2d`	All resources where the given station id does match.
`phenomenon`	`phenomenon=phe_sdjf210`	All resources where the given phenomenon id does match.
`locale`	`locale=de`	Specifies the language as ISO639 code. Subdivisions are respected by an `_` , e.g. `en_US` for American English. The REST-API should consider the following order to detect a locale: 1. User-agent (e.g. header information) 2. `locale` parameter A fallback language cannot be specified as it depends on the data of the service provider.

Note that for `GET` requests the options have to be **URL-encoded** appropriately!

Paging

TODO:
Paging has to be reviewed and is subject to change. Do not rely on paging parameters, format and paging method like HTTP header links.

Resource collections can optionally been paged by the `offset` and `limit` parameters.

Considering the Web Linking (RFC5988) approach the exact location of a page within a collection can be retrieved from response header information.

Paging Example

```
GET /api/v1/stations?offset=4&limit=2 HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Link: </api/v1/stations?offset=4&limit=2>; rel="self"
Link: </api/v1/stations?offset=0&limit=2>; rel="first"; title="first page"
Link: </api/v1/stations?offset=2349&limit=2>; rel="last"; title="last page"
Link: </api/v1/stations?offset=2&limit=2>; rel="prev"; title="previous page"
Link: </api/v1/stations?offset=6&limit=2>; rel="next"; title="next page"

Content omitted for brevity.
```

Search

The query parameter `q` allows simple search over resources. Note that the searching respects the current locale being selected.

Method	Path	Content-Type	Description
`GET`	`/api/v1/search?q=mySearch`	`application/json`	Searches globally over available resources.

Combined search terms are not supported with this API version.

Output Example

A search generates a result collection containing all matching resources as items. An item describes a label where search took place on, a resource type hint and a link to the resource itself.

```
GET /api/v1/search?q=sylt HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK

[
  {
    "id": "sta_a4bff7499d440d9bb67ad73e048c5a47",
    "label": "List_Auf_Sylt_9570070",
    "type": "station",
    "href": "./stations/sta_a4bff7499d440d9bb67ad73e048c5a47"
  },
  {
    "id": "ts_c8ed5ddbb89c19b2e385de23eecbde98",
    "label": "Wasserstand@List_Auf_Sylt_9570070",
    "type": "timeseries",
    "href": "./timeseries/ts_c8ed5ddbb89c19b2e385de23eecbde98"
  },
  {
    "id": "foi_25e09ee3155ab0fbf3e4684ad2edffbf",
    "label": "List_Auf_Sylt_9570070",
    "type": "feature",
    "href": "./features/foi_25e09ee3155ab0fbf3e4684ad2edffbf"
  },
  {
    "id": "pro_e878d48e49ebc2a156e2810470bae9a2",
    "label": "Wasserstand-List_Auf_Sylt_9570070",
    "type": "procedure",
    "href": "./procedures/pro_e878d48e49ebc2a156e2810470bae9a2"
  }
]
```

Common Status Codes

A client shall expect the following HTTP status codes returned by the API.

Code	When
`200`	Status `OK`. All went fine.
`400`	Status `Bad Request`. The request data cannot be read.
`404`	Status `Not Found`. The resource is not available.
`500`	Status `Internal Server Error`. Please contact the webadmin. In some cases an error message will be shown. But in general the server logs will help to trace the problem.

Collections

top

URLs

API calls are triggered by using the following URLs.

Method	Path	Content-Type	Description
`GET`	`/api/v1/`	`application/json`	Lists all resource collections available.

Examples

Simple View

```
GET /api/v1/ HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

Show/Hide Response

Expanded View

```
GET /api/v1/?expanded=true HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

Show/Hide Response

Stations

top

URLs

API calls are triggered by using the following URLs.

Method	Path	Content-Type	Description
`GET`	`/api/v1/stations`	`application/json`	Lists all measured stations available.
`GET`	`/api/v1/stations/{id}`	`application/json`	Lists the measured stations with id `{id}`.

Query Parameters

The collection can be queried by optional filter parameters.

Parameter	Example	Description
`crs`	`crs=EPSG:25832`	The client indicates that input geometries shall be interpreted as being referenced by the given CRS (in this case to 'EPSG:25832' (ETRS89 / UTM zone 32N)). Per default geometries are referenced by the WGS84 CRS (precisely: CRS84, lon/lat order). If `crs` value differs from default, all output geometries are transformed to match the asked CRS. Please note that some CRS switch coordinate axes. Double check axes ordering before interpreting coordinates! Future versions of the API may provide kind of a `strictXY` parameter or similar to give clients more control on this.

`bbox`

```
bbox={
  'll': {
    "type": "Point",
    "coordinates": [<x>,<y>]
  },
  'ur': {
    "type": "Point",
    "coordinates": [<x>,<y>]
  }
}
```

Stations within a given bounding box (in WGS84 by default). Refer to the `crs` parameter if a non-default CRS shall be used.

`near`

```
near={
  "center": {
    "type": "Point",
    "coordinates": [<x>,<y>]
  },
  "radius":500
}
```

Stations around a given point (in WGS84 by default). The radius unit is in kilometers (km). Refer to the `crs` parameter if a non-default CRS shall be used.

`service`	`service=srv_sdlfj2`	All resources where the given service id does match.
`category`	`category=cat_sadf2d`	All resources where the given category id does match.
`phenomenon`	`phenomenon=phe_sdjf210`	All resources where the given phenomenon id does match.

> For this resource further common query parameters can be used.

> Note that for `GET` requests the options have to be **URL-encoded** appropriately!

Examples

Simple Collection

```
GET /api/v1/stations/ HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

Show/Hide Response

> As noted above `expanded=false` is the default. Setting the parameter to `false` explicitly will have the same effect.

Expanded Collection

```
GET /api/v1/stations/?expanded=true HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

Show/Hide Response

> Please refer to the GeoJSON specification if you need more detailed information about the geometry types used.

Item

```
GET /api/v1/stations/sta_a1164b01586abf442ad312211d9f18e/ HTTP/1.1
  Host: sensorweb.demo.52north.org
  Accept: application/json
```

Show/Hide Response

Timeseries

top

URLs

API calls are triggered by using the following URLs.

Method	Path	Content-Type	Description
`GET`	`/api/v1/timeseries`	`application/json`	Lists all timeseries available.
`GET`	`/api/v1/timeseries/{id}`	`application/json`	Lists the metadata of timeseries with id `{id}`.
`GET`	`/api/v1/timeseries/{id}/extras`	`application/json`	Lists extra metadata which may vary from instance to instance. The available metadata fields are listed in the `extras` metadata section of a timeseries. You can query just the ones you are interested in by appending the `fields` query parameter and passing a comma-separated value list to it. For example http://localhost:8080/sensorwebclient-webapp/api/v1/timeseries/{id}/extras?fields=license
`GET`	`/api/v1/timeseries/{id}/{interval}`	`image/png`	Returns an image of timeseries with id `{id}` for the given `{interval}`. Allowed interval values are • `lastDay`

- `lastWeek`
- `lastMonth`

Timeseries can be pre-rendered by a regular task. If pre-rendering is enabled for a given timeseries depends on the API provider as it is a configuration issue.

If a timeseries has no pre-rendered chart a `404` (Resource not found) error is returned. In particular cases this is also true when the chart is currently being re-rendered.

Request Data

Timeseries data is actually not modelled as a resource itself. Data requests can be triggered by appending the `getData` verb.

Method	Path	Content-Type	Description
`GET`	`/api/v1/timeseries/{id}/getData`	`application/json`	Lists the raw data of timeseries with id `{id}`.
`GET`	`/api/v1/timeseries/{id}/getData`	`image/png`	Draws a graph of the data of timeseries with id `{id}`.
`POST`	`/api/v1/timeseries/getData`	`application/json`	Request the metadata for multiple timeseries. The requested timeseries ids have to be contained in the POST request. Have a look at the POST request example or check out the request schema how a POST request shall look like to get raw data for multiple timeseries.
`POST`	`/api/v1/timeseries/getData`	`image/png` `application/pdf`	Exports the given stack of timeseries to the given output format. The requested timeseries ids have to be contained in the POST request. Each timeseries can be styled (chart type, color, widths, etc) by passing style options. Have a look at the POST request example or check out the request schema how a POST request shall look like to render multiple timeseries.

Query Parameters

Timeseries data can be queried with the following filter parameters.

Parameter	Example	Description
`expanded`	`expanded=true`	The `expanded` parameter works for collection data and also when requesting raw timeseries data. Timeseries may provide additional data which changes over time (and therefore is not core metadata anymore). Best example are reference values which can be used to provide better interpretation to actual data values.
`timespan`	`timespan=PT12H/2013-08-06`	An ISO8601 formatted period. Only backslash formatted values are supported.
`width`	`width=200`	If `image/png` is requested, `width` determines the width in pixels.
`height`	`height=200`	If `image/png` is requested, `height` determines the width in pixels.

`style`

```
style={
  "chartType": "line",
  "properties": {
    "color": "#2f2f2f",
    "lineType": "solid",
    "width": 2
  }
}
```

Style options how the timeseries shall be rendered if an `image/png` shall be the output.

A more detailed explanation about styling can be found under the styles section.

Parameter	Example	Description
`legend`	`legend=true`	Indicates if a legend shall be rendered directly on the image output (if `image/png` shall be the output).
`generalize`	`generalize=true`	Indicates that timeseries data shall be generalized. See genralization section for details.
`force_latest_values`	`force_latest_values=true`	The `force_latest_values` parameter can be used to force last value requests on a collection of timeseries. Please note that this enforcement (in combination with `expanded=true`) may result is long response times as the last value has to be requested for each timeseries of the requested collection. Use this enforcement only on filtered queries.
`format`	`format=highcharts`	Controls the output format of raw timeseries data. This avoids converting between different data formats on client side which uses a 3rd party chart rendering API. Currently, supported formats: • `tvp`: Time value pairs (the default) • `highcharts`: Highcharts series data format. Refer to the Format section for detailed information how the formats will look like.
`base64`	`base64=true`	Indicates binary images shall be encoded to a base64 string on server side. This makes it easily possible to display images from JavaScript (you would need a byte parsing library otherwise). Here is an example how to embed encoded base64 string: `<img src='data:image/png;base64," + encodedString + "' />`
`service`	`service=srv_sdlfj2`	All resources where the given service id does match.
`category`	`category=cat_sadf2d`	All resources where the given category id does match.
`phenomenon`	`phenomenon=phe_sdjf210`	All resources where the given phenomenon id does match.

> For this resource further common query parameters can be used.

> Note that for `GET` requests the options have to be **URL-encoded** appropriately!

Examples

Simple Collection

```
GET /api/v1/timeseries HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

Show/Hide Response

> As noted above `expanded=false` is the default. Setting the parameter to `false` explicitly will have the same effect.

Expanded Collection

```
GET /api/v1/timeseries/?expanded=true HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

Show/Hide Response

Simple Item

```
GET /api/v1/timeseries/ts_c8ed5ddbb89c19b2e385de23eecbde98 HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

Show/Hide Response

Raw Data of a single timeseries

```
GET /api/v1/timeseries/ts_c8ed5ddbb89c19b2e385de23eecbde98/getData?expanded=true HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

Show/Hide Response

> Timeseries data can get huge quickly depending on how often new values are measured and how long data is stored in the corresponding data backend! Naive clients can bother data providers when trying to request huge datasets by accident. Therefore data requests may be limited to a maximum timespan (per configuration).
>
> However, most probably consuming huge datasets may also not be of client's interest. Have a look at the paging documentation how to get data in smaller chunks.

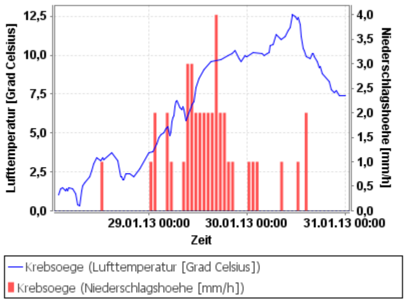
Raw Data of multiple timeseries

```
POST /api/v1/timeseries/getData HTTP/1.1
Host: sensorweb.demo.52north.org
Content-Type: application/json
Accept: application/json

{
  "timespan": "PT6h/2013-08-16TZ",
  "timeseries": [
    "ts_c8ed5ddbb89c19b2e385de23eecbde98",
    "ts_ad3edeff973ab62e39f76c14f95d1e82"
  ]
}
```

Show/Hide Response

Image Data of combined timeseries

```
POST /api/v1/timeseries/getData HTTP/1.1
Host: sensorweb.demo.52north.org
Content-Type: application/json
Accept: image/png

{
  "legend": true,
  "timespan": "P0Y0M3D/2013-01-31TZ",
  "width": 400,
  "height": 300,
  "language": "de",
  "grid": true,
  "styleOptions": {
    "ts_eafa3af4e61db76b367746980d149b7e": {
      "chartType": "line",
      "properties": {
        "color": "#0000FF",
        "lineType": "solid",
        "width": 1
      }
    },
    "ts_92385cd6fb670e886cab8e542185e847": {
      "chartType": "bar",
      "properties": {
        "color": "#2f2f2f",
        "interval": "byHour",
        "width": 0.7
      }
    }
  }
}
```



> If you are interested in the PNG output you can either parse it from your favorite programming language. For a quick review you can use Curl from command line (adapt parameters as needed, e.g. if you want a PDF report instead):
>
> ```
> curl -X POST -d 'PASTE IN HERE YOUR POST REQUEST' \
> -H "content-type:application/json" -H "accept:image/png" \
> http://localhost:8080/sensorwebclient-webapp/api/v1/timeseries/getData > img.png
> ```

Diagram Styles

top

The following diagram types are available and can be set via `style` options:

- line charts
- bar charts

Styles can be used whereever a chart is rendered. It does not matter if you request an image or a report (which embeds the timeseries chart).

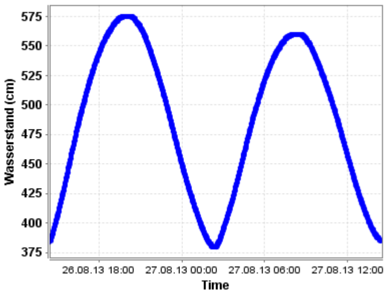
Line Charts

The following style properties can be set.

Property	Description
`lineType`	The line style. Possible values are `solid`, `dotted` or `dashed`
`width`	The thickness of a line as integer, dash gap, point size (dependend on the chart type to be rendered).

`color`	A 6-digit hex color value, e.g. `#5f5f5f`.

Example

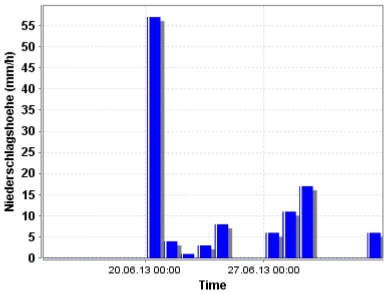


```
{
  "chartType": "line",
  "properties": {
    "type": "dashed",
    "width": 4,
    "color": "#0000ff"
  }
}
```

Bar Charts

Property	Description
`interval`	The time period a bar shall represent. Possible values are `byDay`, `byWeek`, `byMonth`.
`width`	Value between 0 and 1, which defines the bar width in percent (`width=1` means maximum width, i.e. bar next to bar).
`color`	A 6-digit hex color value, e.g. `#5f5f5f`.

Example



```
{
  "chartType": "bar",
  "properties": {
    "interval": "byDay",
    "width": 0.8,
    "color": "#0000ff"
  }
}
```

Data Formats

top

Raw data can be requested in a specific format. This can be useful if you work with a specific chart API and want to avoid to convert data outout from one format to another. Possible formats are:

- `tvp` (the default)
- `highchart`

To retrieve extra reference values (if available for that timeseries) valid for the requested timespan just add `expand=true`.

> Serving arbitrary formats is limited. Most probably you have to combine timeseries metadata and the actual data differently within the used API. Please refer to the actual data output so that it can be used as intended by the 3rd party API.

Default Format

The format returns timeseries data as time-value tuples.

Example (Single Timeseries)

```
{
  "values": [
    {
      "timestamp": 1376524800000,
      "value": 433.0
    },
    {
      "timestamp": 1376524860000,
      "value": 432.4
    },
    {
      "timestamp": 1376524920000,
      "value": 432.0
    },
    {
      "timestamp": 1376524980000,
      "value": 431.1
    }
  ]
}
```

Example (Multiple Timeseries)

```
{
  "ts_ad3edeff973ab62e39f76c14f95d1e82": {
    "values": []
  },
  "ts_c8ed5ddbb89c19b2e385de23eecbde98": {
    "values": [
      {
        "timestamp": 1376589600000,
        "value": 546
      },
      {
        "timestamp": 1376589660000,
        "value": 546.6
      },
      {
        "timestamp": 1376589720000,
        "value": 547
      }
    ]
  }
}
```

> Please note that when adding `expanded=true`, reference values will be added as a timeseries on their own. You can track reference values with the corresponding identifiers available from the timeseries metadata.

Highchart Format

The format returns timeseries data in Highcharts series format. To add metadata or a readable series name you will have to replace the timeseriesId with a readable label taken from the Timeseries metadata.

Example (Single Timeseries)

```
{
  "name": "ts_c8ed5ddbb89c19b2e385de23eecbde98",
  "data": [
    [
      1376524800000,
      433.3
    ],
    [
      1376524860000,
      432.4
    ],
    [
      1376524920000,
      432.1
    ]
  ]
}
```

Example (Multiple Timeseries)

```
[
  {
    "name": "ts_ad3edeff973ab62e39f76c14f95d1e82",
    "data": []
  },
  {
    "name": "ts_c8ed5ddbb89c19b2e385de23eecbde98",
    "data": [
      [
        1376589600000,
        546
      ],
      [
        1376589660000,
        546.6
      ],
      [
        1376589720000,
        547
      ],
      [
        1376589780000,
        548
      ]
    ]
  }
]
```

Generalization

top

Depending on sampling resolution and timespan timeseries data can be huge. Generalizing data can make sense in more than just a low bandwidth use case (e.g. smoothing the curve).

The API supports two generalization algorithms which can be enabled by `generalize=true`. Default generalization behaviour is set to `false` (may be configurable in future). More algorithms can be added easily.

Douglas-Peucker Downsampling

- Downsamples timeseries values by using a threshold value (Details)
- Parameters
    - `generalize=true`
    - `generalizing_algorithm=dp`
    - `tolerance_value=<double-value>` (default is `0.1`)

Largest-Triangle-Three Bucket Downsampling

- Downsamples to a fix amount of output values (Details)
- Parameters
    - `generalize=true`
    - `generalizing_algorithm=lttb`
    - `threshold=<int-value>` (default is `200`)

Parameters

top

Services

URLs

API calls are triggered by using the following URLs.

Method	Path	Content-Type	Description
`GET`	`/api/v1/services`	`application/json`	Lists all services available.
`GET`	`/api/v1/services/{id}`	`application/json`	Lists the service with id `{id}`.

Query Parameters

> For this resource common query parameters can be used.

Examples

Collection

```
GET /api/v1/services/ HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

[
  {
    "id": "srv_823sd",
    "label": "PEGELONLINE"
  },
  {
    "id": "srv_8asdfd",
    "label": "Wupperverband"
  }
]
```

Item

```
GET /api/v1/services/srv_823sd HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

{
  "id": "srv_1a5bde0a6d702f193f7be463402ec12f",
  "label": "PEGELONLINE SOS",
  "serviceUrl": "http://pegelonline.wsv.de/webservices/gis/gdi-sos",
  "version": "1.0.0",
  "type": "SOS",
  "supportsFirstLatest": true,
  "quantities": {
    "categories": 10,
    "phenomena": 10,
    "stations": 551,
    "timeseries": 740
  }
}
```

An expanded view gives you a bit more metadata about a service instance. It counts the amount of available resources and gives you information about the serivce's capabilities, e.g. if first and latest value requests are supported.

Categories

URLs

API calls are triggered by using the following URLs.

Method	Path	Content-Type	Description
`GET`	`/api/v1/categories`	`application/json`	Lists all measured categories available.
`GET`	`/api/v1/categories/{id}`	`application/json`	Lists the measured categories with id `{id}`.

Query Parameters

> For this resource common query parameters can be used.

Examples

Collection

```
GET /api/v1/categories HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

[
  {
    "id": "cat_h7234u",
    "label": "Water"
  },
  {
    "id": "cat_eddfnq",
    "label": "Air"
  }
]
```

Item

```
GET /api/v1/categories/cat_eddfnq HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

{
  "id": "cat_eddfnq",
  "label": "Air",
  "service": {
    "id": "../services/srv_823sd",
    "label": "PEGELONLINE"
  }
}
```

Phenomena

URLs

API calls are triggered by using the following URLs.

Method	Path	Conetnet-Type	Description
`GET`	`/api/v1/phenomena`	`application/json`	Lists all measured phenomena available.
`GET`	`/api/v1/phenomena/{id}`	`application/json`	Lists the measured phenomenon with id `{id}`.

Query Parameters

> For this resource common query parameters can be used.

Examples

Collection

```
GET /api/v1/phenomena HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

[
  {
    "id": "phe_adsf221",
    "label": "Lufttemperatur"
  },
  {
    "id": "phe_adsf221",
    "label": "Wasserstand"
  }
]
```

Item

```
GET /api/v1/phenomena/phe_adsf221 HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

{
  "id": "phe_adsf221",
  "label": "Wasserstand",
  "service": {
    "id": "../services/srv_823sd",
    "label": "PEGELONLINE"
  }
}
```

Unstable/Legacy

top

> This section describe non public functionlity which may be either unstable or legacy to be kept for backward compatibility.

Offerings

URLs

API calls are triggered by using the following URLs.

Method	Path	Content-Type	Description
`GET`	`/api/v1/offerings`	`application/json`	Lists all offerings available.
`GET`	`/api/v1/offerings/{id}`	`application/json`	Lists the offering with id `{id}`.

Query Parameters

Except the `Resources` collection all other collections can be queried/filtered by using the following query parameters. Each parameter is optional and can only contain one value. Multiple parameters are combined to an `AND` query. A resource may specify further query parameters to control/filter output. Special query parameters are described in each section.

Parameter	Example	Description
`feature`	`feature=foi_adf092e`	All resources where the given feature id does match.
`procedure`	`procedure=pro_asd23`	All resources where the given procedure id does match.

> For this resource further common query parameters can be used.

> Note that for `GET` requests the options have to be **URL-encoded** appropriately!

Examples

Collection

```
GET /api/v1/offerings HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

[
  {
    "id": "off_h7234u",
    "label": "ABFLUSS_ROHDATEN"
  },
  {
    "id": "off_eddfnq",
    "label": "WINDGESCHWINDIGKEIT"
  }
]
```

Item

```
GET /api/v1/offerings/off_h7234u HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

{
  "id": "off_h7234u",
  "label": "ABFLUSS_ROHDATEN"
  "service": "../services/srv_823sd"
}
```

Features

URLs

API calls are triggered by using the following URLs.

Method	Path	Content-Type	Description
`GET`	`/api/v1/features`	`application/json`	Lists all features available.
`GET`	`/api/v1/features/{id}`	`application/json`	Lists the feature with id `{id}`.

Query Parameters

Except the `Resources` collection all other collections can be queried/filtered by using the following query parameters. Each parameter is optional and can only contain one value. Multiple parameters are combined to an `AND` query. A resource may specify further query parameters to control/filter output. Special query parameters are described in each section.

Parameter	Example	Description
`offering`	`offering=off_fdoi23`	All resources where the given offering id does match.
`procedure`	`procedure=pro_asd23`	All resources where the given procedure id does match.

> For this resource further common query parameters can be used.

> Note that for `GET` requests the options have to be **URL-encoded** appropriately!

Examples

Collection

```
GET /api/v1/features HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

[
  {
    "id": "foi_h7234u",
    "label": "Bramsche"
  },
  {
    "id": "foi_adsf221",
    "label": "Suelfeld"
  }
]
```

Item

```
GET /api/v1/features/foi_h7234u HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

{
  "id": "foi_h7234u",
  "label": "Bramsche",
  "service": {
    "id": "../services/srv_823sd",
    "label": "PEGELONLINE"
  }
}
```

Procedures

URLs

API calls are triggered by using the following URLs.

Method	Path	Content-Type	Description
`GET`	`/api/v1/procedures`	`application/json`	Lists all procedures available.
`GET`	`/api/v1/procedures/{id}`	`application/json`	Lists the procedure with id `{id}`.

Query Parameters

Except the `Resources` collection all other collections can be queried/filtered by using the following query parameters. Each parameter is optional and can only contain one value. Multiple parameters are combined to an `AND` query. A resource may specify further query parameters to control/filter output. Special query parameters are described in each section.

Parameter	Example	Description
`offering`	`offering=off_fdoi23`	All resources where the given offering id does match.
`feature`	`feature=foi_adf092e`	All resources where the given feature id does match.
`rawFormat`	`rawFormat=http://www.opengis.net/sensorml/2.0`	Returns the requested item encoded in the queried format. The supported raw output formats are listed in the expanded resource (`expanded` query parameter) as `rawFormats` list. The example returns OGC SensorML 2.0 encoded procedure.

> For this resource further common query parameters can be used.

> Note that for `GET` requests the options have to be **URL-encoded** appropriately!

Examples

Collection

```
GET /api/v1/procedures HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

[
  {
    "id": "pro_h7234u",
    "label": "Year.Cmd.MHQ"
  },
  {
    "id": "pro_adsf221",
    "label": "Bad Hersfeld 1_Q#TagMittel"
  }
]
```

Item

```
GET /api/v1/procedures/pro_adsf221 HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

{
  "id": "pro_h7234u",
  "label": "Wasserstand-Berlin-Ploetzensee_Op_586640",
  "service": {
    "id": "../services/srv_823sd",
    "label": "PEGELONLINE"
  }
}
```

Timeseries

Query Parameters

Two new parameter are implemented:

Parameter	Example	Description
`status_intervals`	`status_intervals=true`	Information about the current status of a timeseries. A client should receive metadata how the last measured value can be interpreted by means of defined status "scales".
`rendering_hints`	`rendering_hints=true`	Responds the timeseries resource with a rendering hing, which describes how the timeseries can look like. The styling options are similar to the previous descripted Styles.
`rawFormat`	`rawFormat=http://www.opengis.net/om/2.0`	Returns the requested item encoded in the queried format. The supported raw output formats are listed in the expanded resource (`expanded` query parameter) as `rawFormats` list. The example returns OGC O&M 2.0 encoded observations.

Examples

Item

```
GET api/v1/timeseries/ts_2395d0548435a7ab701f63e2032eaaeb?status_intervals=true HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

{
  id: "ts_2395d0548435a7ab701f63e2032eaaeb",
  label: "Lufttemperatur@Barmen_Wupperverband",
  uom: "Grad Celsius",
  station: {
    properties: {
      id: "sta_254b3e34bbefac4119acd69ec76e647f",
      label: "Barmen_Wupperverband"
    },
    geometry: {
      coordinates: [
        7.204441970508681,
        51.266086785330224
      ],
      type: "Point"
    },
    type: "Feature"
  },
  referenceValues: [ ],
  firstValue: {
    timestamp: 1053934200000,
    value: 15.4
  },
  lastValue: {
    timestamp: 1371057750000,
    value: 20.3
  },
  parameters: {
    service: {
      id: "srv_738111ed219f738cfc85be0c8d87843c",
      label: "Wupperverband SOS"
    },
    offering: {
      id: "off_4f4c3d989f9cee86eb36abadaec6fcba",
      label: "Luft"
    },
    feature: {
      id: "foi_d6bea596d0a9b87cf8f7813acd480080",
      label: "Barmen_Wupperverband"
    },
    procedure: {
      id: "pro_3eb970971e126124ad0a731c2ddd4f30",
      label: "Lufttemperatur_Barmen_Wupperverband"
    },
    phenomenon: {
      id: "phe_1965ee24af8af96270002b4a434a4250",
      label: "Lufttemperatur"
    },
    category: {
      id: "cat_b43c06883f4f439241b59f28033830ad",
      label: "Lufttemperatur"
    }
  },
  statusIntervals: [
    {
      lower: "20.0",
      name: "over 20",
      color: "#FF0000"
    },
    {
```

```
      lower: "15.0",
      upper: "20.0",
      name: "15 - 20",
      color: "#BF003F"
    },
    {
      lower: "10.0",
      upper: "15.0",
      name: "10 - 15",
      color: "#7F007F"
    },
    {
      lower: "5.0",
      upper: "10.0",
      name: "5 - 10",
      color: "#3F00BF"
    },
    {
      lower: "0.0",
      upper: "5.0",
      name: "0 - 5",
      color: "#0000FF"
    }
  ]
}
```

Item

```
GET api/v1/timeseries/ts_5c305e686027a27b5c09b0e4d7397bdd?rendering_hints=true HTTP/1.1
Host: sensorweb.demo.52north.org
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

{
  id: "ts_5c305e686027a27b5c09b0e4d7397bdd",
  label: "Niederschlagshoehe@RS-Herrmannsmuehle",
  uom: "mm/h",
  station: {
    properties: {
      id: "sta_a6b8ef2480c7363d440fc299dc8841eb",
      label: "RS-Herrmannsmuehle"
    },
    geometry: {
      coordinates: [
        7.226317896922862,
        51.19657532705112
      ],
      type: "Point"
    },
    type: "Feature"
  },
  referenceValues: [ ],
  firstValue: {
    timestamp: 1149595200000,
    value: 0
  },
  lastValue: {
    timestamp: 1372892400000,
    value: 0
  },
  parameters: {
    service: {
      id: "srv_738111ed219f738cfc85be0c8d87843c",
      label: "Wupperverband SOS"
    },
    offering: {
      id: "off_f1f50366e5af4c86d57628adc67064a6",
      label: "Niederschlag"
    },
    feature: {
      id: "foi_abaac0b717c43a395b547fb9c70029d2",
      label: "RS-Herrmannsmuehle"
    },
    procedure: {
      id: "pro_40aaea656ee27e215eb6b150525875fb",
      label: "Niederschlagshoehe_RS-Herrmannsmuehle__Stundensumme"
    },
    phenomenon: {
      id: "phe_39b6c8bd9f50fc1892b407c9268cf557",
      label: "Niederschlagshoehe"
    },
    category: {
      id: "cat_dc4c2df7e0450c33a5477c6b8dafab4d",
      label: "Niederschlagshoehe"
    }
  },
  renderingHints: {
    chartType: "bar",
    properties: {
      interval: "byHour",
      width: "0.8",
      color: "#0000ff"
    }
  }
}
```

Design and Content 52°North.
Content licensed under CC BY 3.0.
Glyphicons Free licensed under CC BY 3.0.

top